Yes! Your understanding is 100% correct.

In fact, your idea turns the "AI Chatbot" into a standard CRUD (Create, Read, Update, Delete) + Logic problem. This is the safest way to pass your FYP because you are fully in control of the code, and you are not "plagiarizing" external AI libraries.

Here is the blueprint of how to implement your specific idea using Spring Boot and MongoDB.

1. The Database Design (MongoDB)
You need a collection (table) to store the knowledge. Let's call it ChatRule.

Java Model (ChatRule.java): Instead of one keyword per answer, it is smarter to have a list of keywords for one answer.

Example: Keywords ["leave", "holiday", "vacation"] all point to -> "Go to the HR Portal."

Java

```
@Document(collection = "chat_rules")
public class ChatRule {
    @Id
    private String id;

    // Example: ["apply leave", "take holiday", "vacation"]
    private List<String> keywords;

    // Example: "You can apply for leave in the Employee Dashboard."
    private String response;

    // Standard Getters and Setters
}
```

2. Admin Side: Managing the "Brain"
This is exactly as you described. The Admin teaches the bot.

What you need to code:

Frontend (HTML/JS): A page with a list of existing rules and a button "Add New Rule."

Backend (Controller): Standard CRUD endpoints.

Controller Endpoints:

GET /api/admin/rules -> Retrieves all rules (to show in the list).

POST /api/admin/rules -> Adds a new rule (Admin types keywords and the answer).

PUT /api/admin/rules/{id} -> Updates a rule (e.g., fixing a typo in an answer).

DELETE /api/admin/rules/{id} -> Deletes a rule (if it's no longer valid).

3. Employee Side: Using the Chatbot
This is where the "logic" happens. The employee doesn't see the list of keywords; they just see a chat box.

What you need to code:

Frontend (HTML/CSS/JS):

A simple chat interface (using for bubbles).

When the user clicks "Send", Javascript takes the text and sends a POST

request to your backend.

Backend (The Matching Logic):

The Logic Flow (Service Layer): This is the code you will show your supervisor to prove you wrote the algorithm.

Java

```
public String processUserQuestion(String userQuestion) {
    // 1. Fetch all rules from MongoDB
    List<ChatRule> allRules = repository.findAll();

    // 2. Normalize user input (convert to lowercase)
    String input = userQuestion.toLowerCase();

    // 3. Loop through every rule in the database
```

```
    for (ChatRule rule : allRules) {
        // Loop through the list of
keywords for this specific rule
        for (String keyword :
rule.getKeywords()) {

            // CHECK: Does the user's
sentence contain this keyword?
            if
(input.contains(keyword.toLowerCase()
)) {
                return rule.getResponse(); //
Match found! Return the answer.
            }
        }
    }

    // 4. No match found
    return "I am sorry, I don't understand.
Please contact HR manually.";
}
```

4. Visualizing the Frontend (Since No Bootstrap)

Since you cannot use Bootstrap, you have to write the CSS for the chat bubbles yourself. It is actually quite simple.

CSS Concept for your Chat Page:

Container: A div with overflow-y: scroll (so it scrolls like WhatsApp).

User Message: A div floated to the right, background color Green.

Bot Response: A div floated to the left, background color Grey.

Why this structure is perfect for your specific constraints:
Dynamic: You don't need to restart the server to add new questions. The Admin just adds them to MongoDB via the webpage, and the Bot instantly "knows" the new answer.

Pure Code: You rely on String.contains() and Java Loops. This is basic Java, completely allowed.

Full Marks: You are demonstrating Database Design (storing arrays of strings), API Development (CRUD), and Algorithm Logic (Search/Match).